

Ecole

백준 문제 풀이

2025.10.10

Python

원하는 정수 찾기(이진탐색)



핵심

문제[029] 원하는 정수 찾기

시간 제한 2초 | 난이도 실버 IV | 백준 온라인 저지 1920번

N개의 정수 $A[1], A[2], \dots, A[N]$ 이 주어져 있을 때 이 안에 X라는 정수가 존재하는지 알아내는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 자연수 $N(1 \leq N \leq 100,000)$, 그다음 줄에 N개의 정수 $A[1], A[2], \dots, A[N]$ 이 주어진다. 그다음 줄에 $M(1 \leq M \leq 100,000)$, 그다음 줄에 M개의 수들이 주어지는데, 이 수들이 A 안에 존재하는지 알아내면 된다. 모든 정수의 범위는 -231 보다 크거나 같고, 231보다는 작다.

↑ 출력

M개의 줄에 답을 출력한다. 존재하면 1, 존재하지 않으면 0을 출력한다.

예제 입력 1

```
5 // 데이터 개수
4 1 5 2 3
5 // 찾아야 할 숫자 개수
1 3 7 9 5
```

예제 출력 1

```
1
1
0
0
1
```



이진 탐색 구현

두 인접한 데이터의 크기를 비교해
정렬하는 방법

시간 복잡도는 $O(n^2)$ 으로 다른 정렬
알고리즘보다 속도가 느림

현재 값과 다음 값을 비교해 swap
을 통해 정렬

```
import sys
input = sys.stdin.readline

n = int(input())
nList = list(map(int, input().split()))
nList.sort()

m = int(input())
mList = list(map(int, input().split()))

for m in mList:
    left = 0
    right = n-1

    while left <= right:
        mid = (left + right) // 2
        if m > nList[mid]:
            left = mid + 1
        elif m < nList[mid]:
            right = mid - 1
        else:
            left = mid
            right = mid
            break

    if left == mid and right == mid:
        print(1)
    else:
        print(0)
```


Java, JS

원하는 정수 찾기(이진탐색)



Java

```
public static void main(String[] args) throws IOException { new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    int n = Integer.parseInt(br.readLine());
    int[] nList = new int[n];
    StringTokenizer st = new StringTokenizer(br.readLine());
    for (int i = 0; i < n; i++) {
        nList[i] = Integer.parseInt(st.nextToken());
    }
    // 이진 탐색을 위해 배열을 정렬
    Arrays.sort(nList);

    int m = Integer.parseInt(br.readLine());
    st = new StringTokenizer(br.readLine());
    StringBuilder sb = new StringBuilder();

    for (int i = 0; i < m; i++) {
        int target = Integer.parseInt(st.nextToken()); // 찾아야 할 숫자

        int left = 0;
        int right = n - 1;
        boolean found = false; // 찾았는지 여부를 저장하는 변수

        // 이진 탐색 시작
        while (left <= right) {
            int mid = (left + right) / 2;

            if (target > nList[mid]) {
                left = mid + 1;
            } else if (target < nList[mid]) {
                right = mid - 1;
            } else { // 숫자를 찾은 경우
                found = true;
                break;
            }
        }

        if (found) {
            sb.append(1).append("\n");
        } else {
            sb.append(0).append("\n");
        }
    }

    System.out.print(sb.toString());
}
```

JavaScript

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

// 입력 데이터 파싱
const n = parseInt(input[0]);
const nList = input[1].split(' ').map(Number);
const m = parseInt(input[2]);
const mList = input[3].split(' ').map(Number);

// 이진 탐색을 위해 배열을 오름차순으로 정렬
// sort() 메서드에 비교 함수를 전달해야 숫자를 제대로 정렬할 수 있음
nList.sort((a, b) => a - b);

const results = []; // 결과값을 저장할 배열

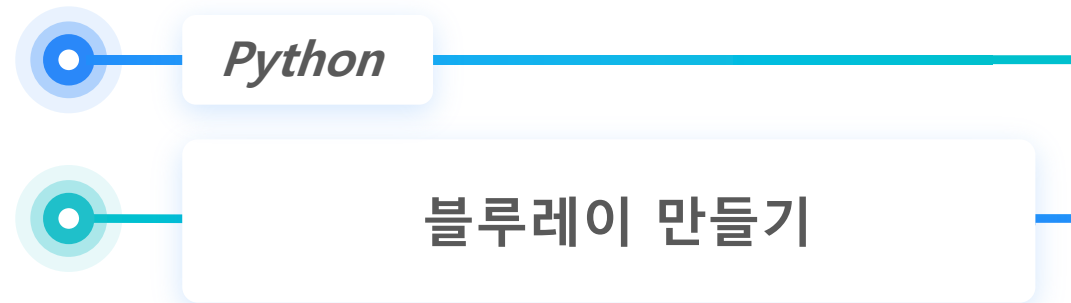
// 찾아야 할 숫자 목록(mList)을 순회
for (const target of mList) {
    let left = 0;
    let right = n - 1;
    let found = false; // 찾았는지 여부를 저장하는 변수

    // 이진 탐색 시작
    while (left <= right) {
        // 중간 인덱스 계산 (소수점 이하 버림)
        let mid = Math.floor((left + right) / 2);

        if (target > nList[mid]) {
            left = mid + 1;
        } else if (target < nList[mid]) {
            right = mid - 1;
        } else { // 숫자를 찾은 경우
            found = true;
            break;
        }
    }

    results.push(found ? 1 : 0);
}

console.log(results.join('\n'));
```



**문제[030] 블루레이 만들기**

시간 제한 2초 | 난이도 실버 I | 백준 온라인 저지 2343번

강토는 자신의 기타 레슨 동영상을 블루레이로 만들어 판매하려고 한다. 블루레이에는 총 N 개의 레슨이 들어가는데, 블루레이를 녹화할 때 레슨의 순서가 바뀌면 안 된다. 순서가 뒤바뀔 때는 레슨의 흐름이 끊겨 학생들이 혼란에 빠질 수 있기 때문이다. 즉, i, j 번 레슨을 같은 블루레이에 녹화하려면 i 와 j 사이의 모든 레슨도 같은 블루레이에 녹화해야 한다.

강토는 이 블루레이가 얼마나 팔릴지 아직 알 수 없어 제작 개수를 가급적 줄이려고 한다. 강토는 고민 끝에 M 개의 블루레이에 모든 기타 레슨 동영상을 녹화하기로 했다. 이때 블루레이의 크기(녹화할 수 있는 길이)는 최소, M 개의 블루레이는 모두 같은 크기로 만들려고 한다.

강토의 각 레슨의 길이가 분 단위(자연수)로 주어질 때 가능한 블루레이의 크기 중 최솟값을 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 레슨의 수 N ($1 \leq N \leq 100,000$)과 M ($1 \leq M \leq N$), 2번째 줄에 강토의 기타 레슨의 길이가 레슨 순서대로 분 단위로(자연수)로 주어진다. 각 레슨의 길이는 10,000분을 넘지 않는다.

↑ 출력

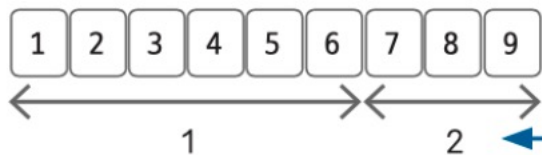
1번째 줄에 블루레이 크기 중 최솟값을 출력한다.

예제 입력 1

```
9 3 // 레슨 수, 블루레이 개수
1 2 3 4 5 6 7 8 9
```

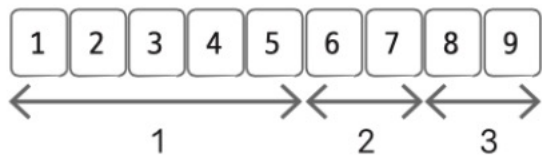
예제 출력 1

```
17
```

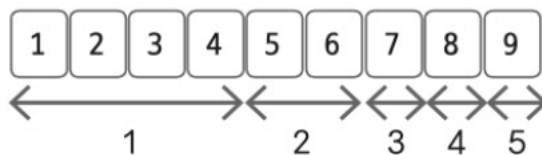


크기 27인 블루레이 3개로 모든 레슨 저장 가능
 $\Rightarrow \text{end} = 27 - 1 = 26$

← 사용된 블루레이 개수



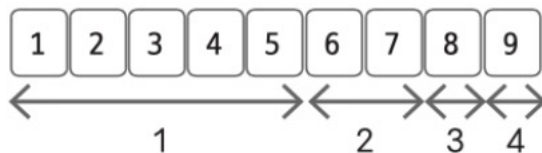
크기 17인 블루레이 3개로 모든 레슨 저장 가능
 $\Rightarrow \text{end} = 17 - 1 = 16$



크기 12인 블루레이 3개로 모든 레슨 저장 불가능
 $\Rightarrow \text{start} = 12 + 1 = 13$

⋮

⋮



크기 16인 블루레이 3개로 모든 레슨 저장 불가능
 $\Rightarrow \text{start} = 16 + 1 = 17$
 $\Rightarrow \text{start} > \text{end}$ 를 만족하여 탐색 종료 후 start 값 출력
 $\therefore 17$

Python

```
import sys
input = sys.stdin.readline

n, m = map(int, input().split())
times = list(map(int, input().split()))

start = max(times)
end = sum(times)

while start <= end:
    mid = (start + end) // 2
    tmp = 0
    count = 0

    for t in times:
        if tmp + t > mid:
            count += 1
            tmp = 0
        tmp += t

    if tmp != 0:
        count += 1

    if count > m:
        start = mid + 1
    else:
        end = mid - 1

print(start)
```

Java, JS

블루레이 만들기



Java

```

public static void main(String[] args) throws IOException {
    new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st = new StringTokenizer(br.readLine());

    int n = Integer.parseInt(st.nextToken());
    int m = Integer.parseInt(st.nextToken());

    int[] times = new int[n];
    int start = 0; // 이진 탐색 시작점 (가장 긴 강의의 길이)
    int end = 0; // 이진 탐색 끝점 (모든 강의 길이의 합)

    st = new StringTokenizer(br.readLine());
    for (int i = 0; i < n; i++) {
        times[i] = Integer.parseInt(st.nextToken());
        end += times[i];
        if (start < times[i]) {
            start = times[i];
        }
    }

    while (start <= end) {
        int mid = (start + end) / 2; // 테스트 해볼 블루레이의 크기
        int count = 0;
        int tmp = 0;

        for (int t : times) {
            if (tmp + t > mid) {
                count++;
                tmp = 0;
            }
            tmp += t;
        }

        if (tmp != 0) count++;

        if (count > m) start = mid + 1;
        else end = mid - 1;
    }

    System.out.println(start);
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const [n, m] = input[0].split(' ').map(Number);
const times = input[1].split(' ').map(Number);

// 이진 탐색의 시작점: 가장 긴 강의 길이
let start = Math.max(...times);
// 이진 탐색의 끝점: 모든 강의 길이의 합
let end = times.reduce((acc, cur) => acc + cur, 0);

while (start <= end) {
    // 테스트해 볼 블루레이의 크기
    let mid = Math.floor((start + end) / 2);
    let count = 0; // 사용된 블루레이 개수
    let tmpSum = 0; // 현재 블루레이에 담긴 강의 길이 합

    for (const t of times) {
        // 현재 블루레이에 더 담을 수 없으면, 새 블루레이를 사용
        if (tmpSum + t > mid) {
            count++;
            tmpSum = 0; // 새 블루레이이므로 합계 초기화
        }
        tmpSum += t;
    }

    // 마지막에 담긴 강의가 있다면 블루레이 개수 +1
    if (tmpSum !== 0) {
        count++;
    }

    // 블루레이 개수가 M보다 많으면, 블루레이 크기를 늘려야 함
    if (count > m) {
        start = mid + 1;
    }

    // 블루레이 개수가 M 이하이면, 크기를 더 줄여볼 수 있음
    else {
        end = mid - 1;
    }
}

// 최적의 크기(최소값)는 start에 저장됨
console.log(start);

```

Python

동전 개수의 최솟값 구하기

**문제[032] 동전 개수의 최솟값 구하기**

시간 제한 1초 | 난이도 실버 Ⅲ | 백준 온라인 저지 11047번

준규가 소유하고 있는 동전은 총 N 종류이고, 각 동전의 개수가 많다. 동전을 적절히 사용해 그 가격의 합을 K 로 만들려고 한다. 이때 필요한 동전 개수의 최솟값을 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 N 과 K ($1 \leq N \leq 10$, $1 \leq K \leq 100,000,000$), 2번째 줄부터 N 개의 줄에 동전의 가격 A_i 가 오름차순으로 주어진다 ($1 \leq A_i \leq 1,000,000$, $A_1 = 1$, $i \geq 2$ 일 때 A_i 는 A_{i-1} 의 배수).

↑ 출력

1번째 줄에 K 원을 만드는 데 필요한 동전 개수의 최솟값을 출력한다.

예제 입력 1

```
10 4200 // 동전 수, 목표 금액
1
5
10
50
100
500
1000
5000
10000
50000
```

예제 출력 1

```
6
```

예제 입력 2

```
10 4790
1
5
10
50
100
500
1000
5000
10000
50000
```

예제 출력 2

```
12
```



제출 번호	아이디	문제	결과	메모리	시간	언어	코드 길이
99257570	zetgi7dk5359	4 11047	맞았습니다!!	9588 KB	96 ms	node.js / 수정	775 B
99257557	zetgi7dk5359	4 11047	맞았습니다!!	14340 KB	104 ms	Java 11 / 수정	963 B
99257288	zetgi7dk5359	4 11047	맞았습니다!!	32412 KB	40 ms	Python 3 / 수정	243 B

그리디 알고리즘이란?

미래는 생각하지 않고, 지금 당장 좋은 것을 고르는 법

최적의 해를 구하는데에 사용되는 근사적인 방법

그리디 알고리즘 사용 예시

- AI의 결정 트리 학습법
- 거스름돈 문제
- 최소 신장 트리
- 다익스트라 알고리즘
- 허프만 코드
- UNION & FIND 알고리즘

```
n, k = map(int, input().split())

coins = []
for _ in range(n):
    coins.append(int(input()))

coins.sort(reverse=True)
result = 0

for i in coins:
    result += k // i
    k = k % i
    if k == 0:
        break

print(result)
```

Java, JS

동전 개수의 최솟값 구하기



Java

```
public static void main(String[] args) throws IOException { new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st = new StringTokenizer(br.readLine());

    int n = Integer.parseInt(st.nextToken());
    int k = Integer.parseInt(st.nextToken());

    int[] coins = new int[n];
    for (int i = 0; i < n; i++) {
        coins[i] = Integer.parseInt(br.readLine());
    }
    int result = 0;
    // 가장 가치가 큰 동전부터 확인하기 위해 배열을 거꾸로 순회
    for (int i = n - 1; i >= 0; i--) {
        if (coins[i] <= k) {
            result += (k / coins[i]);
            k %= coins[i];
        }
        if (k == 0) break;
    }
    System.out.println(result);
}
```

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

let [n, k] = input[0].split(' ').map(Number);
const coins = input.slice(1).map(Number);

let result = 0;

// 가장 가치가 큰 동전부터 확인 (배열의 끝부터 시작)
for (let i = n - 1; i >= 0; i--) {
    const coin = coins[i];

    // 남은 금액(k)이 현재 동전 가치보다 크거나 같으면
    if (k >= coin) {
        // 현재 동전으로 지불할 수 있는 개수를 계산해서 더함
        result += Math.floor(k / coin);
        // 남은 금액을 갱신
        k %= coin;
    }

    // 남은 금액이 0이 되면 더 이상 계산할 필요가 없으므로 종료
    if (k === 0) {
        break;
    }
}

console.log(result);
```

Python

카드 정렬하기

**문제[033] 카드 정렬하기**

시간 제한 2초 | 난이도 골드 IV | 백준 온라인 저지 1715번

정렬된 두 묶음의 숫자 카드가 있다. 각 묶음의 카드의 개수가 A, B일 때 보통 두 묶음을 합쳐 1개로 만들려면 $A + B$ 번 비교해야 한다. 예를 들어 20장의 숫자 카드 묶음과 30장의 숫자 카드 묶음을 합치려면 50번의 비교가 필요하다.

매우 많은 숫자 카드 묶음이 책상 위에 놓여 있다고 가정해 보자. 이들을 두 묶음씩 골라 서로 합쳐 나가면 고르는 순서에 따라 비교 횟수가 달라진다. 예를 들어 10장, 20장, 40장의 묶음이 있다면 10장과 20장을 합친 후 합친 30장 묶음과 40장을 합치면 $(10 + 20) + (30 + 40) = 100$ 번의 비교가 필요하다. 그러나 10장과 40장을 합친 후 합친 50장 묶음과 20장을 합치면 $(10 + 40) + (50 + 20) = 120$ 번의 비교가 필요하므로 덜 효율적이다.

N개의 숫자 카드 묶음의 각각의 크기가 주어질 때 최소한 몇 번의 비교가 필요한지를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 N이 주어진다($1 \leq N \leq 100,000$). 그다음 N개의 줄에 걸쳐 숫자 카드 묶음의 각각의 크기가 주어진다. 숫자 카드 묶음의 크기는 1,000보다 작거나 같은 양의 정수다.

↑ 출력

1번째 줄에 최소 비교 횟수를 출력한다.

예제 입력 1

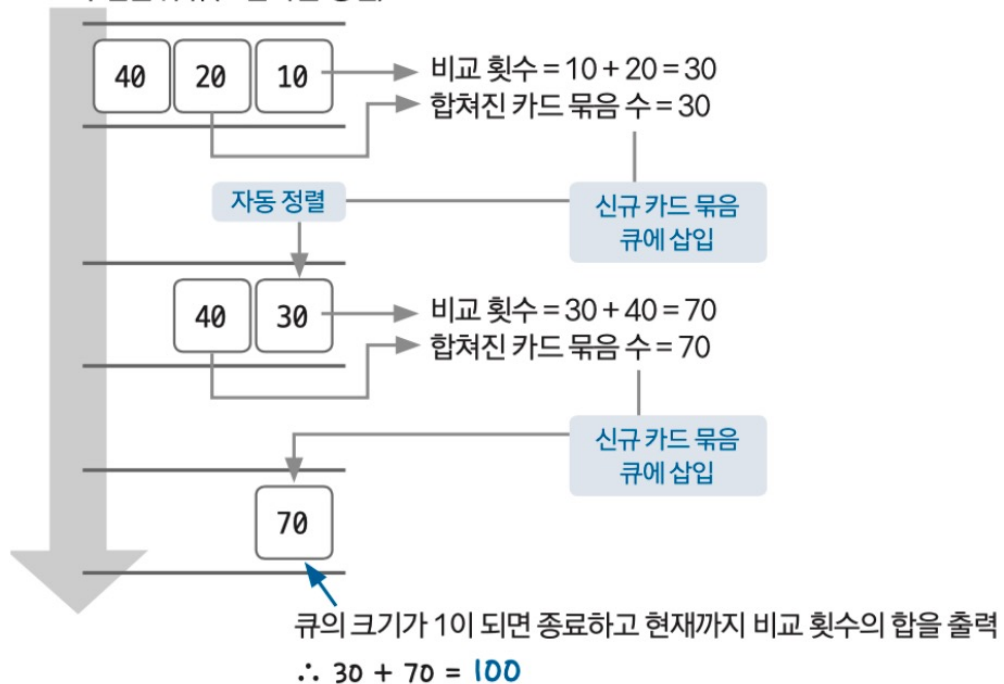
```
3 // 카드 묶음 수
10
20
40
```

예제 출력 1

```
100
```



우선순위 큐(오름차순 정렬)



```
import heapq
import sys
input = sys.stdin.readline

n = int(input())
cards = []
result = 0

for _ in range(n):
    heapq.heappush(cards, int(input()))

if len(cards) == 1:
    print(0)
else:
    while cards:
        if len(cards) == 2:
            print(result + sum(cards))
            break
        tmp = heapq.heappop(cards) + heapq.heappop(cards)
        heapq.heappush(cards, tmp)
        result += tmp
```

Java, JS

카드 정렬하기



PriorityQueue

- 우선순위 큐
- 들어가는 순서와 상관없이 우선순위가 높은 데이터가 먼저 나가는 자료구조

Java

```
public static void main(String[] args) throws IOException {  
    new *  
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
    int n = Integer.parseInt(br.readLine());  
  
    // 우선순위 큐 사용(들어가는 순서와 상관없이 우선순위가 높은 데이터가 먼저 나가는 자료구조)  
    PriorityQueue<Integer> pq = new PriorityQueue<>();  
    for (int i = 0; i < n; i++) {  
        int data = Integer.parseInt(br.readLine());  
        pq.add(data);  
    }  
  
    int data1 = 0;  
    int data2 = 0;  
    int sum = 0;  
  
    while (pq.size() != 1) {  
        data1 = pq.remove();  
        data2 = pq.remove();  
        sum += data1 + data2;  
        pq.add(data1 + data2);  
    }  
  
    System.out.println(sum);  
}
```



JS

카드 정렬하기 - 정답

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

/**
 * 최소 힙 (Min Heap) 클래스 구현
 */
class MinHeap {
  constructor() {
    // 힙을 저장할 배열
    this.heap = [];
  }

  // 힙의 크기를 반환
  size() {
    return this.heap.length;
  }

  // 두 값을 교체하는 메서드
  _swap(idx1, idx2) {
    [this.heap[idx1], this.heap[idx2]] = [this.heap[idx2], this.heap[idx1]];
  }

  // 새로운 값을 힙에 추가
  push(value) {
    this.heap.push(value);
    this._bubbleUp();
  }

  // 가장 작은 값(루트)을 제거하고 반환
  pop() {
    if (this.heap.length === 1) {
      return this.heap.pop();
    }
    const min = this.heap[0];
    this.heap[0] = this.heap.pop();
    this._bubbleDown(0);
    return min;
  }
}
```

```
// 요소를 위로 올려보내며 힙 속성을 유지
_bubbleUp() {
  let index = this.heap.length - 1;
  while (index > 0) {
    const parentIndex = Math.floor((index - 1) / 2);
    if (this.heap[parentIndex] <= this.heap[index]) break;
    this._swap(index, parentIndex);
    index = parentIndex;
  }
}

// 요소를 아래로 내려보내며 힙 속성을 유지
_bubbleDown(index) {
  while (true) {
    let leftChildIndex = 2 * index + 1;
    let rightChildIndex = 2 * index + 2;
    let smallestIndex = index;

    if (leftChildIndex < this.heap.length && this.heap[leftChildIndex] < this.heap[smallestIndex]) {
      smallestIndex = leftChildIndex;
    }
    if (rightChildIndex < this.heap.length && this.heap[rightChildIndex] < this.heap[smallestIndex]) {
      smallestIndex = rightChildIndex;
    }
    if (smallestIndex === index) break;

    this._swap(index, smallestIndex);
    index = smallestIndex;
  }
}
```

```
const n = parseInt(input[0]);
const minHeap = new MinHeap();

for (let i = 1; i <= n; i++) {
  minHeap.push(parseInt(input[i]));
}

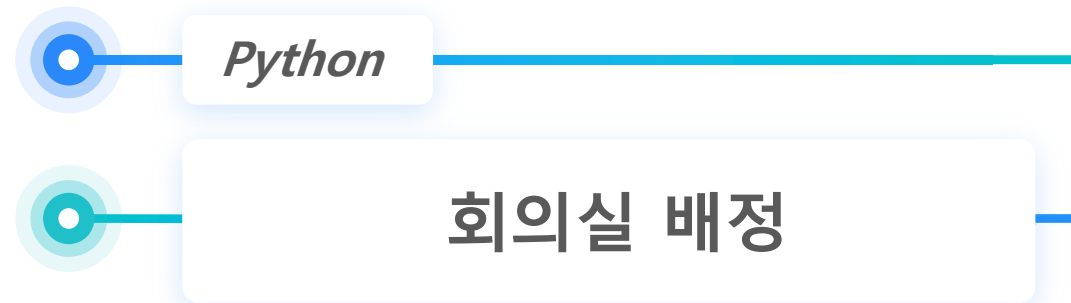
// 카드 묶음이 하나일 경우 비교할 필요가 없으므로 0을 출력
if (n === 1) {
  console.log(0);
} else {
  let totalComparisons = 0;

  // 힙에 묶음이 하나만 남을 때까지 반복
  while (minHeap.size() > 1) {
    // 가장 작은 두 묶음을 꺼내서 합침
    const first = minHeap.pop();
    const second = minHeap.pop();
    const sum = first + second;

    // 합친 횟수(비교 횟수)를 누적
    totalComparisons += sum;

    // 합친 묶음을 다시 힙에 넣음
    minHeap.push(sum);
  }

  console.log(totalComparisons);
}
```



**문제[035] 회의실 배정하기**

시간 제한 2초 | 난이도 실버 I | 백준 온라인 저지 1931번

1개의 회의실에서 N개의 회의를 진행하기 위해 회의실 사용표를 만들려고 한다. 각 회의마다 시작 시간과 끝나는 시간이 주어질 때 회의 시간이 겹치지 않으면서 회의를 가장 많이 진행하려면 최대 몇 번까지 할 수 있는지 알아보자. 단, 회의를 시작하면 중간에 중단할 수 없고, 한 회의를 끝내는 것과 동시에 다음 회의를 시작할 수 있다. 회의의 시작 시간과 끝나는 시간이 같을 수도 있는데, 이때는 시작하자마자 끝나는 것으로 생각하면 된다.

↓ 입력

1번째 줄에 회의의 수 N ($1 \leq N \leq 100,000$), 2번째 줄부터 $N + 1$ 줄까지는 각 회의의 시작 시간과 끝나는 시간이 공백을 사이에 두고 주어진다. 시작 시간과 끝나는 시간은 -1보다 작거나 같은 자연수 또는 0이다.

↑ 출력

1번째 줄에 진행할 수 있는 회의의 최대 개수를 출력한다.

예제 입력 1

```
11 // 회의 개수
1 4
3 5
0 6
5 7
3 8
5 9
6 10
8 11
8 12
2 13
12 14
```

예제 출력 1

```
4
```

1. 현재 회의의 종료 시간이 빠를수록 다음 회의와 겹치지 않게 시작하는 게 유리하다.
2. 종료 시간이 빠른 순서대로 정렬해 겹치지 않는 회의실을 적절하게 선택하여 문제를 해결한다.

[illegible]

총 4개의 회의를
겹치지 않고
진행할 수 있음

```
import sys
input = sys.stdin.readline

n = int(input())
meeting_times = []

for _ in range(n):
    start, end = map(int, input().split())
    meeting_times.append((start, end))

# 회의 시간을 종료 시간 기준 및 시작 시간 기준으로 정렬
meeting_times = sorted(meeting_times, key = lambda x: (x[1], x[0]))

count = 1
# 이전 회의 종료 시간
end_time = meeting_times[0][1]

for i in range(1, n):
    # 다음 회의 시작 시간이 이전 회의 종료 시간보다 크거나 같다면
    if (meeting_times[i][0] >= end_time):
        end_time = meeting_times[i][1]
        count += 1

print(count)
```

Java, JS

회의실 배정



Java

```

public static void main(String[] args) throws IOException { new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    int n = Integer.parseInt(br.readLine());
    int[][] meeting_times = new int[n][2];

    for (int i = 0; i < n; i++) {
        StringTokenizer st = new StringTokenizer(br.readLine());
        meeting_times[i][0] = Integer.parseInt(st.nextToken()); // 시작 시간
        meeting_times[i][1] = Integer.parseInt(st.nextToken()); // 종료 시간
    }

    // 회의 시간을 정렬 (핵심 로직)
    // 1. 종료 시간을 기준으로 오름차순 정렬
    // 2. 종료 시간이 같다면, 시작 시간을 기준으로 오름차순 정렬
    Arrays.sort(meeting_times, new Comparator<int[]>() { new *

        @Override new *
        public int compare(int[] o1, int[] o2) {
            if (o1[1] == o2[1]) {
                return o1[0] - o2[0];
            }
            return o1[1] - o2[1];
        }
    });

    int count = 0;
    int prevEndTime = 0;

    for (int i = 0; i < n; i++) {
        // 이전 회의 종료 시간보다 현재 회의 시작 시간이 크거나 같다면
        if (meeting_times[i][0] >= prevEndTime) {
            prevEndTime = meeting_times[i][1];
            count++;
        }
    }

    System.out.println(count);
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const n = parseInt(input[0]);
const meetingTimes = [];

for (let i = 1; i <= n; i++) {
    meetingTimes.push(input[i].split(' ').map(Number));
}

// 회의 시간을 정렬 (핵심 로직)
// 1. 종료 시간을 기준으로 오름차순 정렬
// 2. 종료 시간이 같다면, 시작 시간을 기준으로 오름차순 정렬
meetingTimes.sort((a, b) => {
    if (a[1] !== b[1]) {
        return a[1] - b[1];
    } else {
        return a[0] - b[0];
    }
});

let count = 0;
let prevEndTime = 0;

for (const meeting of meetingTimes) {
    const startTime = meeting[0];
    const endTime = meeting[1];

    // 이전 회의 종료 시간보다 현재 회의 시작 시간이 크거나 같다면
    if (startTime >= prevEndTime) {
        // 해당 회의를 선택하고, 종료 시간을 갱신
        prevEndTime = endTime;
        count++;
    }
}

console.log(count);

```

Python

최솟값을 만드는 괄호 배치 찾기



핵심

문제[036] 최솟값을 만드는 괄호 배치 찾기

시간 제한 2초 | 난이도 실버 II | 백준 온라인 저지 1541번

세준이는 양수와 +, - 그리고 괄호를 이용해 어떤 수식을 만들었다. 그리고 괄호를 모두 지우고, 다시 괄호를 적절히 넣어 이 수식의 값을 최소로 만들려고 한다. 이렇게 수식의 괄호를 다시 적절하게 배치해 수식의 값을 최소로 만드는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 식이 주어진다. 식은 '0'~'9', '+', '-' 그리고 '(''만으로 이뤄져 있고, 가장 처음과 마지막 문자는 숫자다. 그리고 연속해서 2개 이상의 연산자가 나타나지 않고, 5자리보다 많이 연속되는 숫자는 없다. 수는 0으로 시작할 수 있다. 입력으로 주어지는 식의 길이는 50보다 작거나 같다.

↑ 출력

1번째 줄에 정답을 출력한다.

예제 입력 1

100-40+50+74-30+29-45+43+11

예제 출력 1

-222



1. 가장 작은 최솟값을 만들기 위해서는 가능한 한 큰 수를 빼야 한다.
2. 더하기에 해당하는 부분에 괄호를 쳐서 먼저 모두 계산한 후 빼기를 하면 문제가 해결된다.

```
str = input()
# '-'를 기준으로 수식 분리
nums = str.split('-')

result = sum(map(int, nums[0].split('+')))

for i in range(1, len(nums)):
    subSum = sum(map(int, nums[i].split('+')))
    result -= subSum

print(result)
```

Java, JS

최솟값을 만드는 괄호 배치 찾기



Java

```
public static void main(String[] args) throws IOException { new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    String str = br.readLine();

    // 1. '-'를 기준으로 수식을 분해
    String[] parts = str.split(regex: "-");
    int result = 0;

    // 2. 첫 번째 부분(마이너스 앞부분)의 합을 계산
    String[] firstPartNums = parts[0].split(regex: "\\+");
    for (String numStr : firstPartNums) {
        result += Integer.parseInt(numStr);
    }

    // 3. 두 번째 부분부터는 모두 감소
    for (int i = 1; i < parts.length; i++) {
        String[] subPartNums = parts[i].split(regex: "\\+");
        int subSum = 0;
        for (String numStr : subPartNums) {
            subSum += Integer.parseInt(numStr);
        }
        result -= subSum;
    }

    System.out.println(result);
}
```

JS

```
const fs = require('fs');
const expression = fs.readFileSync('/dev/stdin').toString().trim();

// 1. '-'를 기준으로 수식을 나눔
const parts = expression.split('-');

// 2. 첫 번째 부분의 합을 계산
// split('+') -> map(Number) -> reduce(sum)
const firstSum = parts[0]
    .split('+')
    .map(Number)
    .reduce((acc, cur) => acc + cur, 0);

let result = firstSum;

// 3. 두 번째 부분부터는 모두 빼줌
for (let i = 1; i < parts.length; i++) {
    const subSum = parts[i]
        .split('+')
        .map(Number)
        .reduce((acc, cur) => acc + cur, 0);

    result -= subSum;
}

console.log(result);
```

Python

소수 구하기



빈출

문제[037] 소수 구하기

시간 제한 2초 | 난이도 실버 Ⅲ | 백준 온라인 저지 1929번

M 이상 N 이하의 소수를 모두 출력하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 자연수 M과 N이 빈칸을 사이에 두고 주어진다($1 \leq M \leq N \leq 1,000,000$). M 이상 N 이하의 소수가 1개 이상 있는 입력만 주어진다.

↑ 출력

1줄에 1개씩, 증가하는 순서대로 소수를 출력한다.

예제 입력 1

3 16

예제 출력 1

3
5
7
11
13



1 크기가 $N + 1$ 인 배열을 선언한 후 값은 각각의 인덱스 값으로 채운다.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----

2 1은 소수가 아니므로 삭제한다.

X	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----

3 2부터 N 의 제곱근까지 값을 탐색한다.

값이 인덱스 값이면 그대로 두고, 그 값의 배수를 탐색해 0으로 변경한다.

X	2	3	X	5	X	7	X	9	X	11	X	13	X	15	X
X	2	3	X	5	X	7	X	X	X	11	X	13	X	X	X

```
import math
import sys
input = sys.stdin.readline

def isPrim(num):
    if num == 1:
        return False

    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True

m, n = map(int, input().split())

for i in range(m, n+1):
    if isPrim(i):
        print(i)
```

Java, JS

소수 구하기



Java

```
public static void main(String[] args) throws IOException { new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st = new StringTokenizer(br.readLine());

    int m = Integer.parseInt(st.nextToken());
    int n = Integer.parseInt(st.nextToken());

    StringBuilder sb = new StringBuilder();
    for (int i = m; i <= n; i++) {
        if (isPrime(i)) {
            sb.append(i).append("\n");
        }
    }
    System.out.println(sb.toString());
}

public static boolean isPrime(int num) { 1 usage new *
    if (num < 2) {
        return false;
    }
    // 2부터 해당 수의 제곱근까지 반복
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) {
            return false;
        }
    }
    return true;
}
```

JS

```
const fs = require('fs');
const [m, n] = fs.readFileSync('/dev/stdin').toString().trim().split(' ').map(Number);

// 소수 판별 함수
function isPrime(num) {
    if (num < 2) { // 1은 소수가 아님
        return false;
    }
    // 2부터 해당 수의 제곱근까지 반복
    for (let i = 2; i * i <= num; i++) {
        if (num % i === 0) {
            return false; // 나누어 떨어지면 소수가 아님
        }
    }
    return true;
}

const results = [];
for (let i = m; i <= n; i++) {
    if (isPrime(i)) {
        results.push(i);
    }
}

console.log(results.join('\n'));
```

Python

오일러 피 함수 구하기

**문제[041] 오일러 피 함수 구현하기**

시간 제한 1초 | 난이도 골드 I | 백준 온라인 저지 11689번

자연수 n 이 주어졌을 때 $\text{GCD}(n, k) = 1 (1 \leq k \leq n)$ 을 만족하는 자연수의 개수를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 자연수 $n (1 \leq n \leq 10^{12})$ 이 주어진다.

↑ 출력

$\text{GCD}(n, k) = 1 (1 \leq k \leq n)$ 을 만족하는 자연수의 개수를 출력한다.

예제 입력 1

1 // n

예제 출력 1

1

예제 입력 2

5

예제 출력 2

4

예제 입력 3

10

예제 출력 3

4

예제 입력 4

45

예제 출력 4

24

예제 입력 5

99

예제 출력 5

60



- 1 구하고자 하는 범위까지 배열을 생성한 후 2를 선택한다.

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\phi(N)$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- 2 2의 모든 배수마다 $P[i] = P[i] - P[i] / 2$ 연산을 수행해 값을 갱신한다.

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\phi(N)$	1	1	3	2	5	3	7	4	9	5	11	6	13	7	15

- 3 소수 구하기에서 배수를 지우는 부분만 $P[i] = P[i] - P[i] / K$ 로 변경하면 오일러 피 함수를 구현할 수 있다.
탐색을 계속 진행하면서 $N = \phi(N)$ 인 곳(소수)을 찾아 값을 갱신한다.

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\phi(N)$	1	1	2	2	5	2	7	4	6	5	11	4	13	7	10

start index = 3

$N[3]: 3 - (3/3) = 2$ $N[6]: 3 - (3/3) = 2$ $N[9]: 9 - (9/3) = 6$ $N[12]: 6 - (6/3) = 4$ $N[15]: 15 - (15/3) = 10$

- 4 배열이 끝날 때까지 반복한다.

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\phi(N)$	1	1	2	2	4	2	7	4	6	4	11	4	13	7	8

start index = 5

$N[5]: 5 - (5/5) = 4$ $N[10]: 5 - (5/3) = 4$ $N[15]: 10 - (10/5) = 8$

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\phi(N)$	1	1	2	2	4	2	6	4	6	4	11	4	13	6	8

start index = 7

$N[7]: 7 - (7/7) = 6$ $N[14]: 7 - (7/7) = 6$

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\phi(N)$	1	1	2	2	4	2	6	4	6	4	10	4	12	6	8

start index = 11
start index = 13

$N[11]: 11 - (11/11) = 10$ $N[13]: 13 - (13/13) = 12$

```
import sys
input = sys.stdin.readline

n = int(input())
result = n

for i in range(2, int(n**0.5) + 1):
    if n % i == 0:
        while n % i == 0:
            n //= i
            result -= result / i

if n > 1:
    result -= result / n

print(int(result))
```

Java, JS

오일러 피 함수 구하기



Java

```

public static void main(String[] args) throws IOException { new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    long n = Long.parseLong(br.readLine());
    long result = n;

    // 2부터 n의 제곱근까지 반복
    for (long i = 2; i * i <= n; i++) {
        // i가 n의 소인수인지 확인
        if (n % i == 0) {
            // 오일러 피 함수 공식 적용
            // result = result * (1 - 1/i)
            result = result - result / i;

            // 해당 소인수를 모두 제거
            while (n % i == 0) {
                n /= i;
            }
        }
    }

    // 루프가 끝난 후에도 n이 1보다 크다면,
    // 남은 n은 n의 소인수 중 하나임 (제곱근보다 큰 소인수)
    if (n > 1) {
        result = result - result / n;
    }

    System.out.println(result);
}

```

JS

```

const fs = require('fs');
let n = BigInt(fs.readFileSync('/dev/stdin').toString().trim());

let result = n;

// 2부터 n의 제곱근까지 반복
// BigInt는 Math.sqrt를 직접 사용할 수 없으므로, i*i <= n 조건으로 반복
for (let i = 2n; i * i <= n; i++) {
    // i가 n의 소인수인지 확인
    if (n % i === 0n) {
        // 오일러 피 함수 공식 적용
        // result = result * (1 - 1/i)
        result = result - result / i;

        // 해당 소인수를 모두 제거
        while (n % i === 0n) {
            n /= i;
        }
    }
}

// 루프가 끝난 후에도 n이 1보다 크다면,
// 남은 n은 n의 소인수 중 하나임 (제곱근보다 큰 소인수)
if (n > 1n) {
    result = result - result / n;
}

console.log(result.toString());

```

Python

최소 공배수 구하기

**문제[042] 최소 공배수 구하기**

시간 제한 1초 | 난이도 실버 V | 백준 온라인 저지 1934번

두 자연수 A와 B가 있을 때 A의 배수이면서 B의 배수인 자연수를 A와 B의 공배수라고 한다. 이런 공배수 중 가장 작은 수를 최소 공배수라고 한다. 예를 들어 6과 15의 공배수는 30, 60, 90 등이 있으며, 최소 공배수는 30이다. 두 자연수 A와 B가 주어졌을 때 A와 B의 최소 공배수를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 테스트 케이스의 개수 T($1 \leq T \leq 1,000$), 2번째 줄부터 T개의 줄에 걸쳐 A와 B가 주어진다($1 \leq A, B \leq 45,000$).

↑ 출력

1번째 줄부터 T개의 줄에 A와 B의 최소 공배수를 입력받은 순서대로 1줄에 1개씩 출력한다.

예제 입력 1

```
3 // 테스트 케이스 개수
1 45000 // A, B
6 10
13 17
```

예제 출력 1

```
45000
30
221
```




MOD 연산으로 구현하는 유클리드 호제법

- ① 큰 수를 작은 수로 나누는 MOD 연산을 수행한다.
- ② 앞 단계에서의 작은 수와 MOD 연산 결과값(나머지)으로 MOD 연산을 수행한다.
- ③ 나머지가 0이 되는 순간의 작은 수를 최대 공약수로 선택한다.

최소 공배수는 A와 B가 주어졌을 때 $A * B / \text{최대 공약수}$ 를 계산해 구할 수 있다.

```
import sys
input = sys.stdin.readline

t = int(input())

for _ in range(t):
    a, b = map(int, input().split())
    aa, bb = a, b

    while b > 0:
        a, b = b, a%b

    print(aa*bb//a)
```

Java, JS

최소 공배수 구하기



Java

```

public static void main(String[] args) throws IOException {
    new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
    int T = Integer.parseInt(br.readLine());

    for(int i = 0; i < T; i++){
        StringTokenizer st = new StringTokenizer(br.readLine());
        int A = Integer.parseInt(st.nextToken());
        int B = Integer.parseInt(st.nextToken());

        bw.write(str: A*B/findGCD(A, B) + "\n");
    }

    br.close();
    bw.flush();
    bw.close();
}

//최대 공약수 구하기
static int findGCD(int A, int B){
    /*
    GCD(A, B) = GCD(B, R) = GCD(R, R1) = GCD(R1, R2) = ... = GCD(Rn, 0)
    => Rn이 최대 공약수
    */
    while(B != 0){ //B가 0이 되기 전까지 반복(나머지가 0일때까지)
        int R = A%B; //나머지 저장
        A = B; //B를 A에 저장
        B = R; //A에 B를 저장
    }
    return A;
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const t = parseInt(input[0]);

// 유클리드 호제법으로 최대공약수(GCD)를 구하는 함수
function gcd(a, b) {
    while (b > 0) {
        // Python의 a, b = b, a%b 와 동일한 로직
        [a, b] = [b, a % b];
    }
    return a;
}

const results = [];
for (let i = 1; i <= t; i++) {
    const [a, b] = input[i].split(' ').map(Number);

    // 최소공배수 = (a * b) / 최대공약수
    const lcm = (a * b) / gcd(a, b);
    results.push(lcm);
}

console.log(results.join('\n'));

```

Python

$$Ax + By = C$$

**문제[045] $Ax + By = C$**

시간 제한 1초 | 난이도 골드 I | 백준 온라인 저지 21568번

A, B, C가 주어졌을 때 $Ax + By = C$ 를 만족하면서 다음 조건을 만족하는 (x, y) 쌍을 찾으시오.

- x, y는 정수
- $1,000,000,000 \leq x, y \leq 1,000,000,000$

입력

1번째 줄에 정수 A, B, C가 주어진다.

출력

$Ax + By = C$ 를 만족하는 x, y를 공백으로 구분해 출력한다. 문제의 조건을 만족하는 (x, y)가 존재하지 않을 때는 -1을 출력한다.

- $1,000,000 \leq A, B, C \leq 1,000,000$
- $A, B \neq 0$

예제 입력 1

1 2 3 // A B C

예제 출력 1

3 0

예제 입력 2

3 4 5

예제 출력 2

-5 5

예제 입력 3

6 8 3

예제 출력 3

-1



확장유클리드 호제법

아래 방정식은 $c \% \gcd(a, b) = 0$ 인 경우에만 정수해를 가진다.

다시 말해 c 가 a 와 b 의 최대 공약수의 배수인 경우에만 정수해를 가진다.

즉, $ax + by = c$ 가 정수해를 갖게 하는 c 의 최솟값이 $\gcd(a, b)$ 라는 것을 의미한다.

해를 구하고자 하는 방정식

$ax + by = c$ (a, b, c, x, y 는 정수)

```
import sys
sys.setrecursionlimit(10**6)
input = sys.stdin.readline

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return abs(a)

def extended_euclidean(a, b):
    if b == 0:
        return 1, 0

    x_prime, y_prime = extended_euclidean(b, a % b)
    return y_prime, x_prime - (a//b) * y_prime

a, b, c = map(int, input().split())
common_divisor = gcd(a, b)

if c % common_divisor != 0:
    print(-1)
else:
    x0, y0 = extended_euclidean(a, b)
    mok = c // common_divisor
    print(f"{x0 * mok} {y0 * mok}")
```

Java, JS

$$Ax + By = C$$



Java

```

public static void main(String[] args) throws IOException { new *
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st = new StringTokenizer(br.readLine());

    int a = Integer.parseInt(st.nextToken());
    int b = Integer.parseInt(st.nextToken());
    int c = Integer.parseInt(st.nextToken());
    long gcd = gcd(a, b);
    if (c % gcd != 0) {
        System.out.println(-1);
    } else {
        int mok = (int) (c / gcd);
        long[] ret = Excute(a, b);
        System.out.println(ret[0] * mok + " " + ret[1] * mok);
    }
}

private static long[] Excute(long a, long b) { 2 usages new *
    long[] ret = new long[2];
    if (b == 0) {
        ret[0] = 1; ret[1] = 0;
        return ret;
    }
    long q = a / b;
    long[] v = Excute(b, a % b);
    ret[0] = v[1];
    ret[1] = v[0] - v[1] * q;
    return ret;
}

private static long gcd(long a, long b) { 1 usage new *
    while (b != 0) {
        long tmp = a % b;
        a = b;
        b = tmp;
    }
    return Math.abs(a);
}

```

JS

```

const fs = require('fs');
const [a, b, c] = fs.readFileSync('/dev/stdin').toString().trim().split(' ').map(Number);

function gcd(a, b) {
    while (b !== 0) {
        [a, b] = [b, a % b];
    }
    return Math.abs(a);
}

function extendedEuclidean(a, b) {
    // 베이스 케이스: b가 0이면, gcd(a, 0) = a
    // a*1 + 0*y = a 이므로 x=1, y=0
    if (b === 0) {
        return [1, 0];
    }

    // 재귀 호출을 통해 b, a % b 에 대한 해 [x', y']를 구함
    const [xPrime, yPrime] = extendedEuclidean(b, a % b);

    // 현재 단계의 해 [x, y]를 계산
    // x = y'
    // y = x' - (a / b) * y'
    const x = yPrime;
    const y = xPrime - Math.floor(a / b) * yPrime;

    return [x, y];
}

// 1. a와 b의 최대공약수를 구함
const commonDivisor = gcd(a, b);

// 2. 해가 존재하는지 확인 (C가 최대공약수로 나누어 떨어지는가?)
if (c % commonDivisor !== 0) {
    console.log(-1);
} else {
    // 3. ax + by = gcd(a, b)의 해 [x0, y0]를 구함
    const [x0, y0] = extendedEuclidean(a, b);

    // 4. C를 만들기 위한 배율(mok)을 계산
    // 이미 나누어 떨어짐을 확인했으므로 Math.floor는 생략 가능
    const mok = c / commonDivisor;

    // 5. 최종 해를 계산하여 출력
    console.log(`${x0 * mok} ${y0 * mok}`);
}

```


감사합니다!